

borderclr

PCS ANALYZER

Plugin Distribution Guide

Version 0.1 | Draft Specification

Document status: Draft specification
Target application: PCS Analyzer
Scope: External plugin authors, internal module developers,
and future maintainers

This document defines the recommended packaging, installation, loading,
and distribution rules for PCS Analyzer plugins.

Contents

1	Core Design Principle	2
2	Terminology	2
3	Standard Installed Plugin Layout	3
4	Single-File Plugin Distribution	3
5	Folder-Based Plugin Distribution	4
6	ZIP Plugin Distribution	4
7	manifest.json Format	5
7.1	Minimum Example	5
7.2	Recommended Full Example	5
7.3	Required Fields	5
7.4	Recommended Optional Fields	6
7.5	Deprecated Aliases	6
7.6	Reserved Fields	6
8	Plugin ID Rules	6
9	Entry File Rules	6
10	Import-Time Safety Rules	7
11	Menu Callback Rules	7
12	Folder Plugin Import Rules	8
13	Standalone Compatibility	8
14	PluginApp API Usage	9
15	State Access Levels	9
16	Dependency Handling	9
17	GUI Integration Rules	9
18	Security Rules	10
19	Update and Reinstallation Policy	10
20	Module Manager Behavior	10
21	Registry Format: plugins/plugins.json	10
22	Loader Requirements	11
23	Recommended Plugin Folder Naming	11
24	Resource and Data File Access	11
25	Versioning Policy	12
26	Compatibility Policy	12
27	Logging and Error Handling	12
28	Distribution Checklist	12
29	Minimal Single-File Plugin Template	13
30	Minimal Folder-Based Plugin Template	14
30.1	Folder Structure	14
30.2	manifest.json	14
30.3	plugin.py	14

31	Common Failure Modes	15
31.1	Plugin appears in Module Manager but not in the main menu	15
31.2	Plugin opens automatically on startup	15
31.3	Relative import fails	15
31.4	Windows refuses to update/delete plugin folder	15
32	Recommended Development Workflow	15
33	Summary of Mandatory Rules	16

1 Core Design Principle

PCS Analyzer plugins should be as independent as possible.

The main application provides only:

- the `Modules` menu
- the plugin loading system
- the Module Manager
- a minimal `PluginApp` API
- optional read-only access to selected PCS Analyzer state

A plugin should not require modification of any core PCS Analyzer source file, including `ui/components.py`, `logic/`, `ui/`, or `main.py`.

A plugin registers itself through a `register(app)` function. The main application loads the plugin and calls:

```
plugin_module.register(plugin_app)
```

The plugin is responsible for adding its own menu entry through the provided `PluginApp` object.

PCS Analyzer supports two user-facing plugin distribution styles:

1. **Single-file Python plugins** (`.py`)
2. **Folder-based plugins**, optionally distributed as `.zip` packages

Internally, all plugins are installed as folder-based packages under:

```
plugins/installed/<plugin_id>/
```

Even when a user installs a single `.py` plugin, PCS Analyzer converts it into the standard installed folder layout during installation.

2 Terminology

Term	Meaning
Plugin	An external module installed under <code>plugins/installed/<plugin_id>/</code> .
Built-in module	A tool shipped as part of the PCS Analyzer core distribution, such as PCS Workbench.
Module Manager	GUI used to install, enable, disable, and remove external plugins.
PluginApp	Minimal API wrapper passed to plugins by PCS Analyzer.
Registry	The <code>plugins/plugins.json</code> file that stores installed plugin metadata and enabled state.
Entry file	The Python file loaded by the plugin loader, usually <code>plugin.py</code> .

Both built-in modules and external plugins may be listed under the `Modules` menu. The Module Manager controls only external plugins.

3 Standard Installed Plugin Layout

Every installed plugin must have the following internal structure:

```
plugins/installed/<plugin_id>/├─
  manifest.json└─
  plugin.py
```

For complex plugins, additional folders may be included:

```
plugins/installed/<plugin_id>/├─
  manifest.json└─
  plugin.py└─
  plugin_ui/├─
    __init__.py└─
    main_window.py└─
  plugin_logic/├─
    __init__.py└─
    models.py└─
    fitting.py└─
  resources/├─
    icon.png└─
  examples/└─
    sample_data.csv
```

The plugin loader reads `manifest.json`, finds the entry file, imports it, and calls `register(app)`.

4 Single-File Plugin Distribution

Single-file plugins are recommended for small utilities, such as simple calculators, data converters, quick plotting tools, small export helpers, and prototype analysis tools.

A single-file plugin may be distributed as a plain `.py` file, for example `evans_calculator.py`. It must contain at minimum a `PLUGIN_INFO` dictionary and a `register(app)` function:

```
PLUGIN_INFO = {
    "id": "evans_calculator",
    "name": "Evans Method Calculator",
    "version": "0.1.0",
    "author": "Author Name",
    "description": "Calculate magnetic susceptibility from Evans method data.",
    "type": "window",
    "standalone": True,
    "dependencies": [],
}

def register(app):
    app.add_menu_item(
        label="Evans Method Calculator...",
        command=lambda: open_plugin(app),
    )
```

```
def open_plugin(app):  
    ...
```

When installed through the Module Manager, PCS Analyzer converts it to a standard folder layout. The generated `manifest.json` is derived from `PLUGIN_INF0`. Users should not need to manually create `manifest.json` for single-file plugins.

5 Folder-Based Plugin Distribution

Folder-based plugins are recommended for larger analysis modules, such as Bleaney/VT shift fitting, VT-NMR line-shape analysis, CShM/coordination geometry analysis, crystal-field fitting utilities, and PCS ensemble or motion analysis tools.

A folder-based plugin should be distributed as a folder containing:

1. a valid `manifest.json`
2. the entry file specified by `manifest.json`, usually `plugin.py`
3. a `register(app)` function in the entry file

6 ZIP Plugin Distribution

For public distribution, complex plugins may be packaged as `.zip` files.

Recommended structure:

```
bleaney_vt.zip└─  
  bleaney_vt/└─  
    manifest.json└─  
    plugin.py└─  
    plugin_ui/└─  
    plugin_logic/└─  
    resources/└─  
    examples/
```

During installation, PCS Analyzer should:

1. Extract the zip to a temporary folder
2. Locate `manifest.json`
3. Validate the plugin ID and entry file
4. Copy the plugin folder to `plugins/installed/<plugin_id>/`
5. Register the plugin in `plugins/plugins.json`
6. Refresh the Modules menu if supported

7 `manifest.json` Format

Every installed folder-based plugin must contain a `manifest.json`. Below is a minimum example followed by a recommended full example.

7.1 Minimum Example

```
{
  "id": "test_plugin",
  "name": "Test_Plugin",
  "version": "0.1.0",
  "author": "Author_Name",
  "description": "Simple_plugin_loading_test.",
  "entry": "plugin.py",
  "type": "window",
  "standalone": true
}
```

7.2 Recommended Full Example

```
{
  "id": "bleaney_vt",
  "name": "Bleaney_VT_Shift_Fitting",
  "version": "0.1.0",
  "author": "Author_Name",
  "description": "Temperature-dependent_paramagnetic_shift_fitting_module.",
  "entry": "plugin.py",
  "type": "window",
  "standalone": true,
  "min_app_version": "1.4.0",
  "dependencies": ["numpy", "scipy", "pandas", "matplotlib"],
  "optional_dependencies": ["pyvista", "pyvistaqt", "imageio"],
  "state_access": "read_only",
  "category": "Paramagnetic_NMR"
}
```

7.3 Required Fields

Field	Description
id	Stable and unique plugin identifier. Use lowercase letters, numbers, and underscores.
name	Display name shown in the Module Manager.
version	Plugin version.
entry	Entry Python file, usually <code>plugin.py</code> .

7.4 Recommended Optional Fields

Field	Description
author	Plugin author.
description	Short plugin description.
type	Plugin type: window, analysis, viewer, export, or tool.
standalone	Whether the plugin can run independently.
min_app_version	Minimum PCS Analyzer version required.
dependencies	Required Python packages.
optional_dependencies	Optional packages used for enhanced functionality.
state_access	Intended access level: none, read_only, or read_write.
category	Display or grouping category.

7.5 Deprecated Aliases

New plugins should use "dependencies" and "optional_dependencies". The following older aliases may be accepted by the installer for compatibility but should not be used in new plugins: "requires" and "optional_requires".

7.6 Reserved Fields

The field "entry_function" is reserved for possible future loader support. Current PCS Analyzer plugin loading is based on `register(app)`. Plugins must not rely on `entry_function` unless the loader is explicitly updated to support it.

8 Plugin ID Rules

The plugin ID must be stable and unique. Use lowercase letters, numbers, and underscores.

Good examples:

```
bleaney_vt      evans_calculator    condon_helper
vt_lineshape    coordination_geometry_analyzer
```

Avoid:

```
Bleaney Module    my plugin!!!    test    plugin1
```

The plugin ID is used as the installation folder name, registry key, import namespace, enable/disable state identifier, and update/reinstall matching key. Changing the plugin ID should be treated as creating a new plugin.

9 Entry File Rules

The entry file is usually `plugin.py`. It must define:

```
def register(app):
    ...
```

For a single-file plugin, relative imports should be replaced by local functions or absolute imports. For a folder-based plugin, see Section 12.

10 Import-Time Safety Rules

Important: This is one of the most critical rules. A plugin must not perform any side effects at import time.

A plugin must not perform any of the following at import time:

- open a GUI window
- call `main()` or `mainloop()`
- start calculations or launch subprocesses
- load large files unnecessarily
- modify PCS Analyzer state
- show message boxes (unless dependency validation explicitly requires it)

Only definitions, constants, lightweight imports, and small helper functions should run at import time.

Incorrect

```
from my_tool.standalone import main
main() # launches immediately on import
```

Correct

```
def open_plugin(app):
    from my_tool.standalone import main
    return main()

def register(app):
    app.add_menu_item(
        label="My Tool...",
        command=lambda: open_plugin(app),
    )
```

11 Menu Callback Rules

Plugin menu callbacks must be deferred using `lambda`.

Correct

```
app.add_menu_item(
    label="My Plugin...",
    command=lambda: open_plugin(app),
)
```

Incorrect

```
app.add_menu_item(
    label="My Plugin...",
    command=open_plugin(app), # called immediately during registration
)
```

12 Folder Plugin Import Rules

Folder-based plugins should use package-style relative imports:

```
from .plugin_ui.main_window import MyWindow
from .plugin_logic.fitting import run_fit
```

This requires the plugin loader to use `submodule_search_locations` and register the module in `sys.modules` before execution:

```
spec = importlib.util.spec_from_file_location(
    module_name, entry_path,
    submodule_search_locations=[str(plugin_dir)],
)
module = importlib.util.module_from_spec(spec)
sys.modules[module_name] = module
spec.loader.exec_module(module)
```

If package-style loading is unavailable, local absolute imports may be used, but this is less robust and may conflict with other plugins.

13 Standalone Compatibility

A plugin may also run as a standalone tool. The standalone entry point must be protected:

```
def run_standalone():
    import tkinter as tk
    root = tk.Tk()
    root.title("My Plugin")
    MyWindow(root)
    root.mainloop()

if __name__ == "__main__":
    run_standalone()
```

Never call standalone code directly at import time.

14 PluginApp API Usage

Plugins should interact with PCS Analyzer through the `PluginApp` wrapper. Common API methods include:

```
app.root          # parent Tk window
app.state         # application state dict (read-only)
app.add_menu_item(label, command) # register menu entry
app.add_separator() # add menu separator
app.get_current_structure()
app.get_delta_exp_values()
app.get_pcs_values_by_id()
app.get_metal_position()
app.refresh_views()
```

Direct modification of `app.state` should be avoided unless the plugin is explicitly designed as an integrated plugin.

15 State Access Levels

Value	Meaning
<code>none</code>	Plugin does not use PCS Analyzer state.
<code>read_only</code>	Plugin reads structures, shifts, PCS values, or settings.
<code>read_write</code>	Plugin may modify state. Use only for tightly integrated plugins.

Most plugins should declare `"state_access": "none"` or `"state_access": "read_only"`.

16 Dependency Handling

List required dependencies in `manifest.json`:

```
"dependencies": ["numpy", "scipy", "matplotlib"],
"optional_dependencies": ["pyvista", "pyvistaqt", "imageio"]
```

Plugins should handle missing optional dependencies gracefully and check for required dependencies at runtime without launching the GUI at import time.

17 GUI Integration Rules

- Use `tk.Toplevel(app.root)` for child windows when running inside PCS Analyzer.
- Do not call `tk.Tk()` or `mainloop()` when running inside PCS Analyzer.
- `tk.Tk()` and `mainloop()` are allowed only in standalone mode.

18 Security Rules

Python plugins can execute arbitrary code on the user's computer. The Module Manager should display an appropriate warning and require confirmation before installing any plugin.

Plugin authors should avoid: unexpected network access, modifying user files without confirmation, hidden subprocess execution, modifying PCS Analyzer core files, and storing credentials or private data.

19 Update and Reinstallation Policy

Updating a plugin may require restarting PCS Analyzer. On Windows, an imported plugin may prevent its folder from being deleted.

Recommended update strategy:

1. Rename the existing folder to a backup (e.g., `bleaney_vt.__old__20260516/`).
2. Copy the new plugin folder into `plugins/installed/<plugin_id>/`.
3. Update `plugins/plugins.json`.
4. Refresh the Modules menu.
5. Attempt to delete the backup folder.
6. If deletion fails, leave a `.delete_on_restart` marker and notify the user.

20 Module Manager Behavior

The Module Manager should support: listing, enabling, disabling, removing, and installing plugins (`.py`, folder, or `.zip`).

Note: Visible in Module Manager $\neq$ successfully loaded into the main menu.

If a plugin appears in the Module Manager but not in the main menu, likely causes include: `enabled` is false, `manifest.json` is missing or invalid, `register(app)` is missing, an exception during import or registration, unsupported relative imports, or missing required dependencies.

21 Registry Format: `plugins/plugins.json`

```
{
  "vt_nmr_analysis": {
    "id": "vt_nmr_analysis",
    "name": "VT-NMR_Analysis",
    "version": "3.2.0",
    "dir": "vt_nmr_analysis",
    "enabled": true
  }
}
```

The registry should be treated as application-managed data. Plugin authors should not ask users to edit it manually except for debugging.

22 Loader Requirements

A robust PCS Analyzer plugin loader should:

1. Read `plugins/plugins.json` and skip disabled plugins
2. Resolve `plugins/installed/<plugin_id>/` and read `manifest.json`
3. Validate the entry file
4. Import using package-style loading and register in `sys.modules`
5. Verify and call `register(app)`
6. Store loading results and errors
7. Avoid crashing the whole application when one plugin fails

```
module_name = f"pcs_plugin_{plugin_id}"
spec = importlib.util.spec_from_file_location(
    module_name, entry_path,
    submodule_search_locations=[str(plugin_dir)],
)
module = importlib.util.module_from_spec(spec)
sys.modules[module_name] = module
spec.loader.exec_module(module)
if not hasattr(module, "register"):
    raise RuntimeError("register(app) function is missing.")
module.register(app)
```

23 Recommended Plugin Folder Naming

Use plugin-local folder names to avoid conflicts with PCS Analyzer core modules.

Recommended: `plugin_ui/`, `plugin_logic/`, `resources/`, `examples/`

Avoid: `ui/`, `logic/`, `core/`, `utils/`

24 Resource and Data File Access

Plugins should resolve resource paths relative to the plugin file:

```
from pathlib import Path

PLUGIN_DIR = Path(__file__).resolve().parent
RESOURCE_DIR = PLUGIN_DIR / "resources"
EXAMPLE_DIR = PLUGIN_DIR / "examples"

# Correct
open(RESOURCE_DIR / "icon.png", "rb")

# Incorrect - do not rely on working directory
open("resources/icon.png")
```

25 Versioning Policy

Plugin versions should follow semantic versioning:

Part	Meaning
MAJOR	Breaking changes.
MINOR	New features without breaking compatibility.
PATCH	Bug fixes.

26 Compatibility Policy

Plugins may declare the minimum required PCS Analyzer version via "min_app_version". Future plugin managers may refuse to load plugins requiring a newer version. Plugins should avoid relying on undocumented internal state keys unless declared as `state_access: "read_write"`.

27 Logging and Error Handling

Plugins should catch user-facing errors and report them clearly:

```
try:
    run_analysis()
except Exception as exc:
    from tkinter import messagebox
    messagebox.showerror("My Plugin", f"Analysis failed:\n\n{exc}")
```

Plugin loaders should catch import and registration errors and continue loading other plugins.

28 Distribution Checklist

Before distributing a plugin, verify all of the following:

- ☐ The plugin has a stable lowercase plugin ID.
- ☐ The plugin defines `PLUGIN_INFO` or `manifest.json`.
- ☐ Folder-based plugins include `manifest.json`.
- ☐ The manifest has `id`, `name`, `version`, and `entry`.
- ☐ The entry file defines `register(app)`.
- ☐ The plugin does not open windows at import time.
- ☐ The plugin does not call `main()` at import time.
- ☐ The plugin does not call `mainloop()` inside PCS Analyzer.
- ☐ Menu callbacks use `command=lambda: open_plugin(app)`.
- ☐ Required dependencies are listed.
- ☐ Missing optional dependencies are handled gracefully.
- ☐ Resource paths are resolved relative to the plugin folder.

- ☐ The plugin does not modify PCS Analyzer core source files.
- ☐ The plugin can be removed without breaking the core application.
- ☐ The plugin was tested after installation through Module Manager.
- ☐ The plugin was tested after disabling and re-enabling.
- ☐ The plugin was tested after reinstalling/updating.

29 Minimal Single-File Plugin Template

```
from __future__ import annotations
import tkinter as tk
from tkinter import ttk

PLUGIN_INFO = {
    "id": "minimal_plugin",
    "name": "Minimal Plugin",
    "version": "0.1.0",
    "author": "Author Name",
    "description": "A minimal PCS Analyzer plugin.",
    "type": "window",
    "standalone": True,
    "dependencies": [],
}

def open_plugin(app):
    win = tk.Toplevel(app.root)
    win.title("Minimal Plugin")
    win.geometry("360x160")
    frame = ttk.Frame(win, padding=16)
    frame.pack(fill="both", expand=True)
    ttk.Label(frame, text="Minimal plugin loaded successfully.").pack(anchor="w")
    return win

def register(app):
    app.add_menu_item(
        label="Minimal Plugin...",
        command=lambda: open_plugin(app),
    )

def run_standalone():
    root = tk.Tk()
    root.title("Minimal Plugin")
    class DummyApp:
        root = root
        state = {}
    open_plugin(DummyApp())
    root.mainloop()

if __name__ == "__main__":
    run_standalone()
```

30 Minimal Folder-Based Plugin Template

30.1 Folder Structure

```
my_analysis_plugin/├─
  manifest.json├─
  plugin.py├─
  plugin_ui/│├─
    __init__.py├─
    main_window.py├─
  plugin_logic/├─
    __init__.py├─
    analysis.py
```

30.2 manifest.json

```
{
  "id": "my_analysis_plugin",
  "name": "My_Analysis_Plugin",
  "version": "0.1.0",
  "author": "Author_Name",
  "description": "Example_folder-based_analysis_plugin.",
  "entry": "plugin.py",
  "type": "analysis",
  "standalone": true,
  "dependencies": ["numpy", "matplotlib"],
  "state_access": "read_only"
}
```

30.3 plugin.py

```
from __future__ import annotations

PLUGIN_INFO = {
    "id": "my_analysis_plugin",
    "name": "My Analysis Plugin",
    "version": "0.1.0",
}

def open_plugin(app):
    from .plugin_ui.main_window import MyAnalysisWindow
    return MyAnalysisWindow(parent=app.root, app=app)

def register(app):
    app.add_menu_item(
        label="My Analysis Plugin...",
        command=lambda: open_plugin(app),
    )

def run_standalone():
    import tkinter as tk
    from .plugin_ui.main_window import MyAnalysisWindow
    root = tk.Tk()
    root.title("My Analysis Plugin")
    MyAnalysisWindow(parent=root, app=None)
    root.mainloop()
```

```
if __name__ == "__main__":  
    run_standalone()
```

31 Common Failure Modes

31.1 Plugin appears in Module Manager but not in the main menu

Likely causes:

- `enabled` is `false` in the registry
- `register(app)` is missing
- Import failed due to missing dependency or relative import error
- Plugin opened itself and crashed during import

Check `state["plugin_load_results"]` or console diagnostics.

31.2 Plugin opens automatically on startup

The menu callback was written as `command=open_plugin(app)` instead of `command=lambda: open_plugin(app)`, or `main()` was called at import time.

31.3 Relative import fails

```
ImportError: attempted relative import with no known parent package
```

Fix: update the loader to use `submodule_search_locations`, or avoid relative imports and use local absolute imports. Package-style relative imports are preferred.

31.4 Windows refuses to update/delete plugin folder

```
PermissionError: [WinError 5] Access is denied
```

Likely causes: plugin is currently imported, plugin window is open, or the OS is indexing files. Close plugin windows, restart PCS Analyzer, and try again. The installer should use backup-folder replacement instead of direct deletion.

32 Recommended Development Workflow

1. Develop as standalone tool first.
2. Add `PLUGIN_INFO`.
3. Add `open_plugin(app)`.
4. Add `register(app)`.
5. Ensure no code runs at import time.
6. Install through Module Manager.
7. Test loading from the Modules menu.
8. Test disable/enable.

9. Test reinstall/update.
10. Package as `.py`, folder, or `.zip`.

33 Summary of Mandatory Rules

A valid PCS Analyzer plugin must obey the following rules:

1. It must have a stable plugin ID.
2. It must define `register(app)`.
3. It must not execute the tool at import time.
4. It must use deferred menu callbacks: `command=lambda: open_plugin(app)`.
5. It must not modify PCS Analyzer core files.
6. It must declare dependencies when distributed publicly.
7. It must handle missing optional dependencies gracefully.
8. It must keep standalone execution behind `if __name__ == "__main__":`.

These rules are intended to keep PCS Analyzer modular, stable, and safe as the plugin ecosystem grows.